

✓ Experiment No. 2: Baseline ANN Model with Experiment Tracking

Student Name

Vinay Yadav [23101B0020]

Practical Name

Baseline Artificial Neural Network (ANN) Model with Experiment Tracking using Crop Yield Prediction Dataset

Resources Required

Software

- Google Colab / Jupyter Notebook
- Python 3.10+
- TensorFlow / Keras
- Pandas
- NumPy
- Scikit-learn
- Matplotlib
- Joblib

Dataset

- Crop Yield Prediction Dataset (`crop_yield(1).csv`)
-

Objectives

- Load and preprocess the Crop Yield Prediction dataset.
 - Prepare the dataset for training using feature encoding and normalization.
 - Build a baseline Artificial Neural Network (ANN) model.
 - Train and evaluate the ANN model using appropriate regression metrics.
 - Perform manual experiment tracking using different hyperparameter configurations.
 - Compare multiple experiments to identify the best-performing model.
 - Save the trained ANN model and experiment log for future use.
-

Theory

Artificial Neural Networks (ANNs) are one of the fundamental deep learning models used to learn complex relationships between input features and target variables. An ANN consists of an input layer, one or more hidden layers, and an output layer. During training, the network learns patterns from the data by adjusting its weights through backpropagation and optimization algorithms such as Adam.

Before training an ANN, the dataset must be preprocessed by encoding categorical variables into numerical values and normalizing numerical features to improve convergence and training performance. Proper preprocessing ensures that the model learns efficiently and produces reliable predictions.

A **baseline ANN model** serves as the initial reference model for evaluating future improvements. Rather than immediately building complex architectures, a simple ANN is trained first to establish a performance benchmark.

Experiment tracking is an important practice in deep learning projects. It involves recording hyperparameters such as learning rate, batch size, and epochs along with the corresponding evaluation metrics. Tracking multiple experiments enables systematic comparison of different model configurations, improves reproducibility, and helps identify the best-performing model.

In this experiment, the Crop Yield Prediction dataset is used to build a baseline ANN model for predicting crop yield. Multiple experiments are conducted using different hyperparameter settings, and their results are stored in an experiment log for comparison. Finally, the best-performing model is saved for future prediction and deployment.

Step 1: Import Required Libraries

```
# Data Manipulation
import pandas as pd
import numpy as np
```

```
# Data Preprocessing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Deep Learning
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout

# Evaluation Metrics
from sklearn.metrics import mean_squared_error, mean_absolute_error

# Experiment Tracking
import datetime
```

Step 2: Load the Crop Yield Dataset

```
# =====
# Step 2.1: Upload Dataset from Local Computer
# =====
```

```
from google.colab import files

uploaded = files.upload()
```

No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
# =====
# Step 2.2: Load the Dataset
# =====
```

```
import pandas as pd

# Get the uploaded file name
file_name = list(uploaded.keys())[0]

# Load the dataset
df = pd.read_csv(file_name)

# Display the first 5 rows
df.head()
```

	Region	Soil_Type	Crop	Rainfall_mm	Temperature_Celsius	Fertilizer_Used	Irrigation_Used	Weather_Condition	Days_t
0	West	Sandy	Cotton	897.077239	27.676966	False	True	Cloudy	
1	South	Clay	Rice	992.673282	18.026142	True	True	Rainy	
2	North	Loam	Barley	147.998025	29.794042	False	False	Sunny	
3	North	Sandy	Soybean	986.866331	16.644190	False	True	Rainy	
4	South	Silt	Wheat	730.379174	31.620687	True	True	Cloudy	

Step 3: Explore the Dataset

```
# Display the first 5 rows
print("First 5 Rows:")
display(df.head())

# Display dataset shape
print("\nDataset Shape:")
print(df.shape)

# Display column names
print("\nColumn Names:")
print(df.columns)

# Display dataset information
print("\nDataset Information:")
df.info()

# Display summary statistics
print("\nSummary Statistics:")
display(df.describe())
```

	Region	Soil_Type	Crop	Rainfall_mm	Temperature_Celsius	Fertilizer_Used	Irrigation_Used	Weather_Condition	Days_
0	West	Sandy	Cotton	897.077239	27.676966	False	True	Cloudy	
1	South	Clay	Rice	992.673282	18.026142	True	True	Rainy	
2	North	Loam	Barley	147.998025	29.794042	False	False	Sunny	
3	North	Sandy	Soybean	986.866331	16.644190	False	True	Rainy	
4	South	Silt	Wheat	730.379174	31.620687	True	True	Cloudy	

```
Column Names:
Index(['Region', 'Soil_Type', 'Crop', 'Rainfall_mm', 'Temperature_Celsius',
       'Fertilizer_Used', 'Irrigation_Used', 'Weather_Condition',
       'Days_to_Harvest', 'Yield_tons_per_hectare'],
      dtype='object')
```

Summary Statistics:

	Rainfall_mm	Temperature_Celsius	Days_to_Harvest	Yield_tons_per_hectare
count	1000000.000000	1000000.000000	1000000.000000	1000000.000000
mean	549.981901	27.504965	104.495025	4.649472
std	259.851320	7.220608	25.953412	1.696572
min	100.000896	15.000034	60.000000	-1.147613
25%	324.891090	21.254502	82.000000	3.417637
50%	550.124061	27.507365	104.000000	4.651808
75%	774.738520	33.753267	127.000000	5.879200
max	999.998098	39.999997	149.000000	9.963372

```
# Input Features
X = df.drop("Yield_tons_per_hectare", axis=1)

# Target Variable
y = df["Yield_tons_per_hectare"]

# Display shapes
print("Features Shape :", X.shape)
print("Target Shape   :", y.shape)

# Display first 5 rows of features
display(X.head())

# Display first 5 target values
display(y.head())
```

Features Shape : (1000000, 9)
Target Shape : (1000000,)

	Region	Soil_Type	Crop	Rainfall_mm	Temperature_Celsius	Fertilizer_Used	Irrigation_Used	Weather_Condition	Days_t
0	West	Sandy	Cotton	897.077239	27.676966	False	True	Cloudy	
1	South	Clay	Rice	992.673282	18.026142	True	True	Rainy	
2	North	Loam	Barley	147.998025	29.794042	False	False	Sunny	
3	North	Sandy	Soybean	986.866331	16.644190	False	True	Rainy	
4	South	Silt	Wheat	730.379174	31.620687	True	True	Cloudy	

Yield_tons_per_hectare

0	6.555816
1	8.527341
2	1.127443
3	6.517573
4	7.248251

dtype: float64

Step 5: Encode Categorical Features

```
# Convert categorical variables into numerical variables
X = pd.get_dummies(X, drop_first=True)

# Display the new shape
print("Encoded Features Shape:", X.shape)

# Display the first 5 rows
display(X.head())
```

Encoded Features Shape: (1000000, 20)

	Rainfall_mm	Temperature_Celsius	Fertilizer_Used	Irrigation_Used	Days_to_Harvest	Region_North	Region_South	Region_W
0	897.077239	27.676966	False	True	122	False	False	True
1	992.673282	18.026142	True	True	140	False	True	False
2	147.998025	29.794042	False	False	106	True	False	False
3	986.866331	16.644190	False	True	146	True	False	False
4	730.379174	31.620687	True	True	110	False	True	False

Step 6: Split the Dataset into Training and Testing Sets

```
# Split dataset into Training (80%) and Testing (20%)
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

# Display the shape of each dataset
print("Training Features :", X_train.shape)
print("Testing Features  :", X_test.shape)
print("Training Target   :", y_train.shape)
print("Testing Target    :", y_test.shape)
```

Training Features : (800000, 20)
Testing Features : (200000, 20)
Training Target : (800000,)
Testing Target : (200000,)

Step 7: Normalize (Scale) the Features

```
# Create StandardScaler object
scaler = StandardScaler()

# Fit and transform the training data
X_train_scaled = scaler.fit_transform(X_train)
```

```
# Transform the testing data
X_test_scaled = scaler.transform(X_test)

# Display the shape of scaled datasets
print("Scaled Training Features:", X_train_scaled.shape)
print("Scaled Testing Features :", X_test_scaled.shape)
```

```
Scaled Training Features: (800000, 20)
Scaled Testing Features : (200000, 20)
```

Step 8: Build the Baseline ANN Model

```
# Create the ANN model
model = Sequential([

    # Input Layer + First Hidden Layer
    Dense(128, activation='relu', input_shape=(X_train_scaled.shape[1],)),

    # Dropout Layer
    Dropout(0.2),

    # Second Hidden Layer
    Dense(64, activation='relu'),

    # Output Layer
    Dense(1)

])

# Display model summary
model.summary()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 128)	2,688
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 64)	8,256
dense_2 (Dense)	(None, 1)	65

```
Total params: 11,009 (43.00 KB)
Trainable params: 11,009 (43.00 KB)
Non-trainable params: 0 (0.00 B)
```

Step 9: Compile the ANN Model

```
# Compile the model
model.compile(
    optimizer='adam',
    loss='mean_squared_error',
    metrics=['mae']
)

print("ANN Model Compiled Successfully!")
```

ANN Model Compiled Successfully!

Step 10: Train the Baseline ANN Model

```
# Train the ANN model
history = model.fit(
    X_train_scaled,
    y_train,
    epochs=20,
    batch_size=32,
    validation_split=0.2,
    verbose=1
)
```

```
Epoch 1/20
20000/20000 — 60s 3ms/step - loss: 0.3266 - mae: 0.4434 - val_loss: 0.2704 - val_mae: 0.4151
Epoch 2/20
20000/20000 — 54s 3ms/step - loss: 0.2674 - mae: 0.4123 - val_loss: 0.3262 - val_mae: 0.4555
Epoch 3/20
20000/20000 — 54s 3ms/step - loss: 0.2631 - mae: 0.4089 - val_loss: 0.3517 - val_mae: 0.4725
```

```

Epoch 4/20
20000/20000 — 54s 3ms/step - loss: 0.2611 - mae: 0.4073 - val_loss: 0.3735 - val_mae: 0.4874
Epoch 5/20
20000/20000 — 57s 3ms/step - loss: 0.2600 - mae: 0.4067 - val_loss: 0.3826 - val_mae: 0.4935
Epoch 6/20
20000/20000 — 53s 3ms/step - loss: 0.2591 - mae: 0.4059 - val_loss: 0.3704 - val_mae: 0.4858
Epoch 7/20
20000/20000 — 86s 3ms/step - loss: 0.2583 - mae: 0.4052 - val_loss: 0.4025 - val_mae: 0.5071
Epoch 8/20
20000/20000 — 53s 3ms/step - loss: 0.2576 - mae: 0.4047 - val_loss: 0.3776 - val_mae: 0.4923
Epoch 9/20
20000/20000 — 85s 3ms/step - loss: 0.2569 - mae: 0.4042 - val_loss: 0.3794 - val_mae: 0.4928
Epoch 10/20
20000/20000 — 54s 3ms/step - loss: 0.2568 - mae: 0.4041 - val_loss: 0.3931 - val_mae: 0.5008
Epoch 11/20
20000/20000 — 55s 3ms/step - loss: 0.2567 - mae: 0.4039 - val_loss: 0.3756 - val_mae: 0.4907
Epoch 12/20
20000/20000 — 57s 3ms/step - loss: 0.2563 - mae: 0.4037 - val_loss: 0.3662 - val_mae: 0.4848
Epoch 13/20
20000/20000 — 54s 3ms/step - loss: 0.2560 - mae: 0.4034 - val_loss: 0.4075 - val_mae: 0.5113
Epoch 14/20
20000/20000 — 53s 3ms/step - loss: 0.2558 - mae: 0.4032 - val_loss: 0.3644 - val_mae: 0.4826
Epoch 15/20
20000/20000 — 54s 3ms/step - loss: 0.2558 - mae: 0.4033 - val_loss: 0.3634 - val_mae: 0.4826
Epoch 16/20
20000/20000 — 54s 3ms/step - loss: 0.2555 - mae: 0.4031 - val_loss: 0.3550 - val_mae: 0.4769
Epoch 17/20
20000/20000 — 54s 3ms/step - loss: 0.2554 - mae: 0.4030 - val_loss: 0.3491 - val_mae: 0.4724
Epoch 18/20
20000/20000 — 56s 3ms/step - loss: 0.2551 - mae: 0.4028 - val_loss: 0.3457 - val_mae: 0.4705
Epoch 19/20
20000/20000 — 53s 3ms/step - loss: 0.2553 - mae: 0.4029 - val_loss: 0.3575 - val_mae: 0.4780
Epoch 20/20
20000/20000 — 57s 3ms/step - loss: 0.2551 - mae: 0.4027 - val_loss: 0.3475 - val_mae: 0.4713

```

Step 11: Evaluate the Model

```

# Evaluate the model on the test dataset
test_loss, test_mae = model.evaluate(X_test_scaled, y_test, verbose=1)

print("Test Loss (MSE):", round(test_loss, 4))
print("Test MAE:", round(test_mae, 4))

```

```

6250/6250 — 12s 2ms/step - loss: 0.3490 - mae: 0.4722
Test Loss (MSE): 0.349
Test MAE: 0.4722

```

Step 12: Manual Experiment Tracking

```

# Create an empty list to store experiment details
experiment_log = []

# Store the first experiment
experiment_log.append({
    "Experiment_ID": "EXP-01",
    "Learning_Rate": 0.001,
    "Epochs": 20,
    "Batch_Size": 32,
    "Test_Loss_MSE": round(test_loss, 4),
    "Test_MAE": round(test_mae, 4)
})

# Display the experiment log
log_df = pd.DataFrame(experiment_log)
display(log_df)

```

	Experiment_ID	Learning_Rate	Epochs	Batch_Size	Test_Loss_MSE	Test_MAE
0	EXP-01	0.001	20	32	0.349	0.4722

Step 13: Run Multiple Hyperparameter Configurations

```

# Create a new ANN model
model_exp2 = Sequential([
    Dense(128, activation='relu', input_shape=(X_train_scaled.shape[1],)),
    Dropout(0.2),
    Dense(64, activation='relu'),
    Dense(1)
])

# Compile the model with a different learning rate

```

```

model_exp2.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.0005),
    loss='mean_squared_error',
    metrics=['mae']
)

# Train the model
history2 = model_exp2.fit(
    X_train_scaled,
    y_train,
    epochs=5,
    batch_size=32,
    validation_split=0.2,
    verbose=1
)

# Evaluate the model
test_loss2, test_mae2 = model_exp2.evaluate(
    X_test_scaled,
    y_test,
    verbose=0
)

# Save experiment details
experiment_log.append({
    "Experiment_ID": "EXP-02",
    "Learning_Rate": 0.0005,
    "Epochs": 5,
    "Batch_Size": 32,
    "Test_Loss_MSE": round(test_loss2,4),
    "Test_MAE": round(test_mae2,4)
})

print("EXP-02 Completed Successfully!")

```

```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape` to `Input` layers.
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/5
20000/20000 — 54s 3ms/step - loss: 0.3464 - mae: 0.4525 - val_loss: 0.2952 - val_mae: 0.4335
Epoch 2/5
20000/20000 — 55s 3ms/step - loss: 0.2682 - mae: 0.4130 - val_loss: 0.2973 - val_mae: 0.4350
Epoch 3/5
20000/20000 — 52s 3ms/step - loss: 0.2623 - mae: 0.4085 - val_loss: 0.3271 - val_mae: 0.4557
Epoch 4/5
20000/20000 — 51s 3ms/step - loss: 0.2604 - mae: 0.4069 - val_loss: 0.3168 - val_mae: 0.4483
Epoch 5/5
20000/20000 — 54s 3ms/step - loss: 0.2592 - mae: 0.4061 - val_loss: 0.2985 - val_mae: 0.4362
EXP-02 Completed Successfully!

```

Step 14: Save Experiment Log (experiment_log.csv)

```

# Convert experiment log into a DataFrame
log_df = pd.DataFrame(experiment_log)

# Display the experiment log
display(log_df)

# Save the experiment log
log_df.to_csv("experiment_log.csv", index=False)

print("Experiment log saved successfully as 'experiment_log.csv'")

```

	Experiment_ID	Learning_Rate	Epochs	Batch_Size	Test_Loss_MSE	Test_MAE
0	EXP-01	0.0010	20	32	0.3490	0.4722
1	EXP-02	0.0005	5	32	0.3002	0.4370

Experiment log saved successfully as 'experiment_log.csv'

Step 15: Compare the Results

```

print("Experiment Comparison:")
display(log_df)

# Find the best experiment based on lowest Test MAE
best_experiment = log_df.loc[log_df["Test_MAE"].idxmin()]

print("\nBest Experiment:")
display(best_experiment)

```

Experiment Comparison:

	Experiment_ID	Learning_Rate	Epochs	Batch_Size	Test_Loss_MSE	Test_MAE
0	EXP-01	0.0010	20	32	0.3490	0.4722
1	EXP-02	0.0005	5	32	0.3002	0.4370

Best Experiment:

	1
Experiment_ID	EXP-02
Learning_Rate	0.0005
Epochs	5
Batch_Size	32
Test_Loss_MSE	0.3002
Test_MAE	0.437

dtype: object

Step 16: Save the Best Model (baseline_ann.keras)

```
# Save the best model
model_exp2.save("baseline_ann.keras")

print("Best ANN Model Saved Successfully!")
```

Best ANN Model Saved Successfully!

```
from google.colab import files

files.download("baseline_ann.keras")
```

```
from google.colab import files

files.download("experiment_log.csv")
```

Conclusion

In this experiment, a baseline Artificial Neural Network (ANN) model was successfully developed using the Crop Yield Prediction dataset. The dataset was preprocessed by encoding categorical features and scaling numerical features before training the model. The trained ANN model was evaluated using regression metrics such as Mean Squared Error (MSE) and Mean Absolute Error (MAE).

Multiple experiments were performed using different hyperparameter configurations, and their results were recorded in an experiment log for comparison. The best-performing configuration was identified based on the evaluation metrics, and the trained model was saved for future use. This experiment demonstrated the importance of establishing a baseline model and maintaining systematic experiment tracking to build reproducible and efficient deep learning workflows.

.